AI Engineering and Claude Architect · Part V, Certification

Certification, Lesson 4

The Six Scenarios and the Four Hands-On Exercises

You'll get four of these six scenarios, and each one tests a predictable set of decisions. This lesson maps them, then walks through the guide's own four preparation exercises, so you can practise what the exam describes.

Module	AI13 of 13
Lesson	4 of 5
Prerequisites	AI13 L1–L3, and the Domain reading
Exam relevance	**All six scenarios and all domains**
Reading time	~35 minutes, plus hands-on time

Contents

1. The Six Scenarios and the Four Hands-On Exercises

Lesson 4 of 5 · Certification

The Six Scenarios and the Four Hands-On Exercises

Key idea

Each scenario is a small set of recurring decisions. Learn the decisions, and a new item is often one you've reasoned through already. Then build at least one exercise yourself, since doing it shows you what reading can't, such as a rule that didn't load because of its scope.

> *Before: the Domain reading. Teaches: a decision map for S1–S6, the traps in each, and a walkthrough of the guide's four exercises, with this course's code. Used by: the mocks.*

The hook: fifteen items about one system

The exam doesn't jump between unrelated topics. It gives you a system, such as a support agent, and asks about many decisions in it. If you've thought about that system as a whole, the second item is easier than the first. This lesson does that thinking ahead of time.

The scenario descriptions and primary domains below are from the guide. The likely decisions and traps are this course's analysis of its objectives, not a statement of what's on the exam.



Figure 1. The guide's primary domains for each scenario, grouped. S1 and S3 are agent systems, S2 and S4 are developer workflows, and S5 and S6 lean on prompts.

S1: Customer Support Resolution Agent

The system: Agent SDK, custom MCP tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`), and a target of 80%+ first-contact resolution while knowing when to escalate.

Decision	Best answer	Trap	Study
Refund before verifying identity	Programmatic prerequisite	Stronger prompt	AI06 L8
Refund over a limit	Hook blocks it, redirects to escalation	Prompt rule	AI06 L8
Customer asks for a human	Escalate immediately	Investigate first	AI06 L8
Angry, but simple issue	Acknowledge and resolve	Escalate on sentiment	AI06 L8
Policy is silent	Escalate	Improvise	AI06 L8
Two customers match	Ask for another identifier	Pick the likeliest	AI06 L8
Wrong tool chosen	Improve tool descriptions	Routing classifier	AI06 L3
Multi-issue message	Decompose, investigate in parallel, unify	Handle only the first	AI06 L8–L9
Handoff	Structured summary: ID, cause, amount, action	Transcript, which humans can't see	AI06 L8
Long sessions forget amounts	Persistent case facts	Summarize more	AI03 L6
Order lookup returns 40 fields	Trim to relevant fields	Bigger budget	AI03 L6
Mixed date formats	PostToolUse normalization	Prompt	AI06 L8, AI13 L2
Permanent failure	Non-retryable structured error	Retry	AI06 L5

80% is a target, not a threshold for autonomy. Don't escalate less just to hit it, as AI06 L9 discusses.

S2: Code Generation with Claude Code

The system: a team using Claude Code for generation, refactoring, debugging and documentation.

Decision	Best answer	Trap	Study
New hire misses standards	Move from `~/.claude/` to project scope	Re-send by chat	AI12 L2
Conventions by file type, everywhere	`.claude/rules/` with `paths` globs	Subdirectory `CLAUDE.md`	AI12 L2
Team command	`.claude/commands/` in version control	User-level command	AI12 L3
Noisy skill output	`context: fork`	Compact afterwards	AI12 L3
Skill needs a parameter	`argument-hint`	Prompt text	AI12 L3
Always-on vs on-demand	`CLAUDE.md` vs skill	Everything in one file	AI12 L3
Architecture or 45-file migration	Plan mode	Direct execution	AI12 L4
Single-file fix with a clear trace	Direct execution	Plan mode	AI12 L4
Output inconsistent	2-3 input/output examples	More prose	AI12 L4
Unfamiliar domain	Interview pattern	Start coding	AI12 L4
Fixes interact	One detailed message	One at a time	AI12 L4
Verbose discovery	Explore subagent	Reading all files	AI12 L1, L4
Which memory loaded?	`/memory`	Guessing	AI12 L2

S3: Multi-Agent Research System

The system: Agent SDK, with a coordinator delegating to search, document analysis, synthesis and report subagents, producing cited reports.

Decision	Best answer	Trap	Study
Report misses subtopics	Coordinator decomposed too narrowly	Blame the search agent	AI06 L6
Subagents communicate	Via coordinator only	Direct agent-to-agent	AI06 L6
Synthesis lacks prior findings	Pass them in its prompt	Assume inheritance	AI06 L6
Speed	Multiple spawning calls in one response	Separate turns	AI06 L6
Coordinator instructions	Goals and quality criteria	Procedural steps	AI06 L6
Search timed out	Local retry, then structured error with partial results	"Search unavailable"	AI06 L5
No results vs failure	Distinguish them	Empty as success	AI06 L5
Keep sources	Structured claim-source mappings	Prose summaries	AI05 L4
Sources disagree	Keep both, with attribution	Pick one	AI05 L4
Different dates, different figures	Require dates	Treat as a contradiction	AI05 L4
Gaps	Coverage annotations	Silent omission	AI06 L5, L6
Synthesis checks one fact	Scoped `verify_fact`	Full search tool	AI06 L3
Research too long	Structured data from upstream agents	Verbose chains	AI03 L6
Crash mid-run	Manifest on resume	Restart everything	AI06 L7

S4: Developer Productivity with Claude

The system: Agent SDK, built-in tools (Read, Write, Bash, Grep, Glob), MCP servers, helping engineers explore unfamiliar code, generate boilerplate and automate tasks.

Decision	Best answer	Trap	Study
Find callers of a function	Grep	Glob, or read everything	AI12 L1
Find `**/*.test.tsx`	Glob	Grep	AI12 L1
Edit text isn't unique	Read + Write	Retry Edit	AI12 L1
Understand a large codebase	Grep entry points, then Read and follow	Read all upfront	AI12 L1
Function re-exported through wrappers	Find exported names, then search each	One-name search	AI12 L1
Agent prefers Grep over a better MCP tool	Improve the MCP description	Remove Grep	AI06 L5
Shared server config	`.mcp.json` with `${VAR}`	Secrets in the file	AI06 L5
Personal experiment	`~/.claude.json`	Project file	AI06 L5
Jira integration	Community server first	Custom server	AI06 L5
Catalog of content	MCP resource	Exploratory tools	AI13 L3
Answers turn generic ("typical patterns")	Scratchpad, subagents, summaries, `/compact`	Keep going	AI06 L7
Resume after code changed	Tell the agent which files changed	Assume it knows	AI06 L7
Prior results stale	New session, structured summary	Resume anyway	AI06 L7
Compare two approaches	`fork_session`	Two full re-analyses	AI06 L7
Legacy test coverage	Map, then prioritize, then adapt	Fixed plan	AI06 L1

S5: Claude Code for Continuous Integration

The system: CI/CD running automated reviews, test generation and PR feedback, with actionable feedback and few false positives.

Decision	Best answer	Trap	Study
Job hangs	`-p`	Longer timeout	AI12 L5
Parse findings	`--output-format json`, `--json-schema`	Regex on text	AI12 L5
Review standards	Project `CLAUDE.md`	Paste in each prompt	AI12 L5
Too many false positives	Explicit categorical criteria	"Be conservative"	AI03 L2
One category erodes trust	Disable it while you improve it	Keep it or drop all	AI03 L2
Severity inconsistent	Criteria with code examples per level	Adjectives	AI03 L2
Reviewing generated code	Independent instance	Same session	AI12 L5
14-file PR, shallow and contradictory	Per-file passes + integration pass	A bigger model	AI12 L5
Re-review after new commits	Include prior findings, report new or unaddressed	Start blind	AI12 L5
Duplicate generated tests	Provide existing tests	Ask it to be careful	AI12 L5
Low-value tests	Standards and fixtures in `CLAUDE.md`	Test count targets	AI12 L5
Pre-merge review	Synchronous API	Batch, for the saving	AI03 L5
Nightly or weekly audit	Batch API	Synchronous	AI03 L5
Dismissal analysis	`detected_pattern`	Reasons in prose	AI03 L3

S6: Structured Data Extraction

The system: extracts from unstructured documents, validates with JSON schemas, handles edge cases and feeds downstream systems.

Decision	Best answer	Trap	Study
Reliable shape	Tool use with a schema	"Return JSON"	AI03 L3
Document type unknown	`tool_choice: "any"`, several schemas	`auto`	AI03 L3
One step must run first	Forced tool, then later turns	Prompt order	AI03 L3
Model invents absent values	Nullable fields	"Don't guess"	AI03 L3
New categories	Enum + "other" + detail, plus "unclear"	A huge enum	AI03 L3
Sums wrong	Validators, retry with errors	Stricter schema	AI03 L3, AI13 L3
Value absent	Stop retrying	More retries	AI03 L3, AI13 L3
Formats vary	Normalization rules, few-shot by layout	More instructions	AI03 L2, L3
Totals conflict	`calculated_total`, `conflict_detected`	Overwrite	AI03 L3
Batch failures	Resubmit by `custom_id`, chunk oversized	Rerun all	AI03 L5
Batch SLA	Compute windows	Assume faster	AI03 L5
97% accuracy overall	Check by type and field	Automate	AI10 L3
Measure high-confidence errors	Stratified random sampling	Trust confidence	AI10 L3
Route review	Calibrated field-level confidence	Raw scores	AI10 L3

The one calculation you should be able to do

The guide's example: a 30-hour SLA with 24-hour batch processing. A document can wait up to the window length, plus up to 24 hours. So the window can't exceed 30 − 24 = 6 hours. The guide's 4-hour windows give 4 + 24 = 28 hours in the worst case, which leaves 2 hours for retries or surprises.

SLA	Batch time	Largest window	With a safety margin
30 h	24 h	6 h	4 h leaves a 2 h margin
36 h	24 h	12 h	Choose below 12 h
24 h	24 h	0 h	Batch can't guarantee it

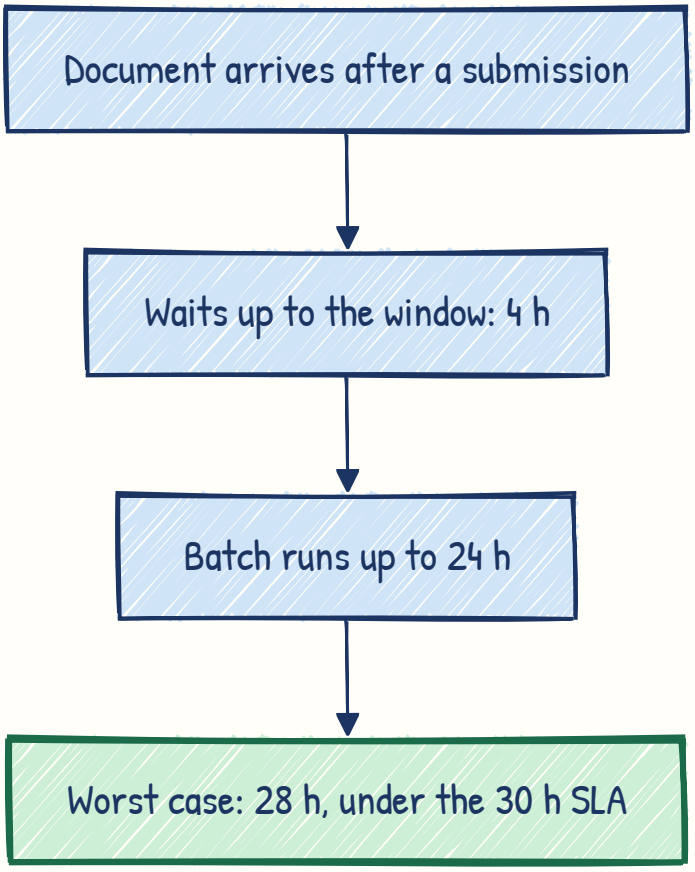


Figure 2. Worst case for a batch window. A document arriving just after a submission waits a full window, then up to 24 hours.

Why reading alone falls short

Study approach	What it misses	What to add
Reading lessons	Scope and hierarchy surprises	Exercise 2
Bank questions only	A system's decisions together	The scenario maps here
Watching videos	Your own failed attempts	Any exercise
Daily Claude Code use	Features you've never needed	Exercise 2's checks

How Anthropic framed it

The guide's section 8 lists four exercises as preparation, and that's the official hands-on advice. What's below is this course's walkthrough of them, not Anthropic's solution.

What's been run in this course. I have no API key, and I didn't spend your subscription, so **none of these walkthroughs has been run end to end against a real model.** Where this course has working parts, the table names them. The steps that need a model are yours to run, and they're low in cost.

The production architecture: Exercise 1, multi-tool agent

Domains 1, 2, 5. **This is a recommended reference design, not a specific company's system.**

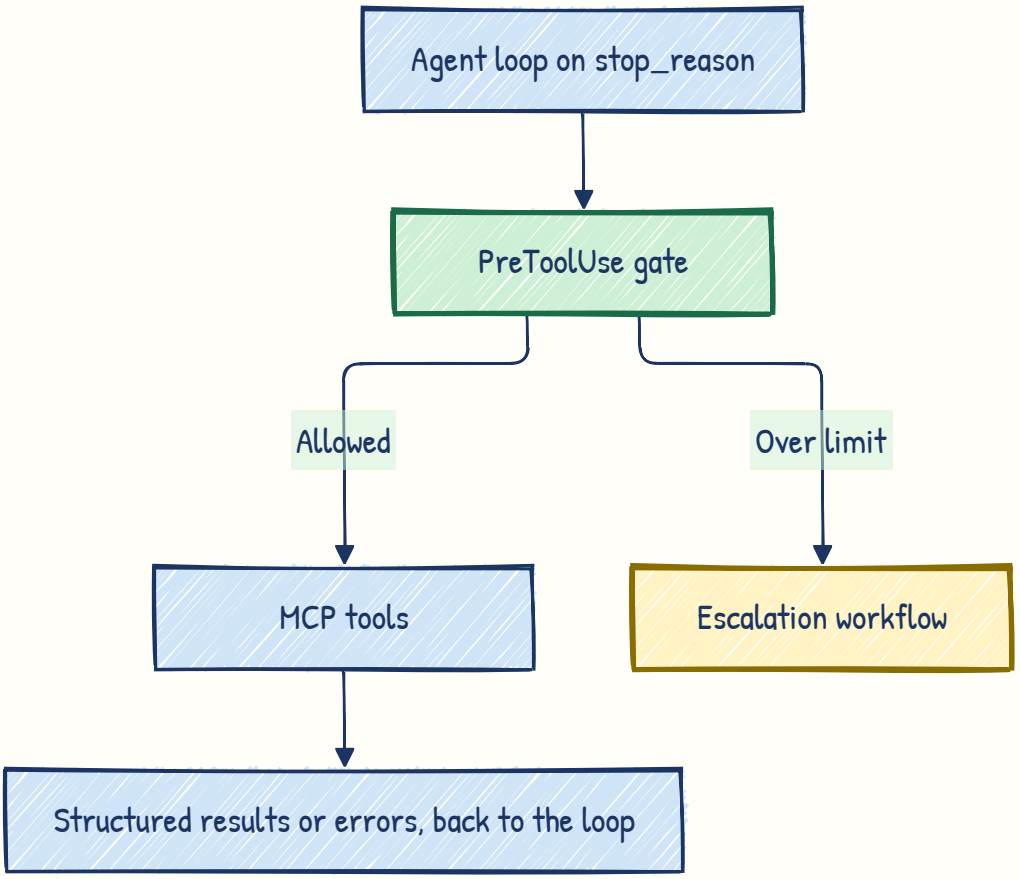


Figure 3. Exercise 1. The hook is the part that makes the rule a guarantee.

Step in the guide	What to build	This course's part	What proves it worked
3-4 tools, two of them similar	`lookup_order` and `search_orders`, with boundaries in the descriptions	AI06 L3, AI08 MCP server	Ambiguous requests pick correctly
Loop on `stop_reason`	`tool_use` continues, `end_turn` ends	AI08 agent loop	Logs show a tool round, then a stop
Structured errors	`errorCategory`, `isRetryable`, a description	AI08 tests	Transient retried, business explained
Business-rule hook	Block over a limit, redirect	`code/ai13-exam/sdk_lab.py`	800 is denied, with no refund
Multi-concern message	Refund plus address change in one message	AI06 L9	One reply covers both

Now try to break it: send a request of $500.01, ask for a human in the first message, and make a tool time out. Each one has a guide-defined correct behavior.

Exercise 2: team workflow in Claude Code

Domains 3, 2. You need Claude Code for this, and a small repository.

```
# .claude/rules/testing.md  (frontmatter)
---
paths: ["**/*.test.*"]
---
```

```
# .claude/skills/analyze/SKILL.md  (frontmatter)
---
name: analyze
description: Maps a module's dependencies. Use before larger changes.
context: fork
allowed-tools: Read, Grep, Glob
argument-hint: <module-path>
---
```

```
{ "mcpServers": { "tracker": {
    "type": "http",
    "url": "https://tracker.example.com/mcp",
    "headers": { "Authorization": "Bearer ${TRACKER_TOKEN}" } } } }
```

The URL is a placeholder, and the JSON goes in `.mcp.json`. AI12's `ConfigLint` checks files like these without a model.

Step	Check it by
Project `CLAUDE.md`	A teammate or second clone gets it. Confirm with `/memory`
Path rules	Edit a matching file and a non-matching one, then see which rules load
Forked skill	Run it, and confirm the verbose work didn't enter the main context
Two MCP scopes	Both servers appear, with `.mcp.json` shared and `~/.claude.json` personal
Plan vs direct	Try a one-file bug, a multi-file migration and a feature with options

Remember the exam's answers while you do this: the guide treats commands and skills as separate, and it says `allowed-tools` restricts. L5 lists how the current docs differ.

Exercise 3: extraction pipeline

Domains 4, 5.

Step	Build	Course part
Schema	Required, optional, nullable, "other" + detail	AI03 L3, AI13 L3
Absent fields	Test that the model returns null	**Needs a model**
Validation retry	Document + failed output + errors, tracking which get resolved	`validation_lab.py` (fake model)
Few-shot	Bibliography and inline citations, table and narrative	AI03 L2
Batch of 100	Submit, handle by `custom_id`, chunk oversized, compute the time	AI03 L5, AI08 Async AI
Review routing	Field confidence, low to human, accuracy by type and field	AI10 L3

Time calculation to do: if 100 documents have a 30-hour SLA, and some fail on the first pass and need resubmission, then two passes in the worst case take 48 hours. So a 24-hour worst case can't guarantee resubmission within that SLA. It's the worst case that's bound by 24 hours, and actual runs may be far shorter. In practice, measure your batches, and keep a synchronous fallback for late failures.

Exercise 4: research pipeline

Domains 1, 2, 5.

```
{
  "claim": "Regional demand rose in 2025",
  "evidence": "Demand grew in 2025, per the annual survey",
  "source": "https://example.com/survey-2025",
  "published": "2026-02-10",
  "status": "contested"
}
```

(A made-up finding, showing the shape only.)

Step	Build	Course part	What proves it worked
Coordinator with subagents	Spawning tool in allowed tools, findings in prompts	AI06 L6, AI13 L2	Subagents succeed without inherited context
Parallel execution	Several calls in one response, timed against sequential	AI06 L6	Time near the slowest, not the sum
Structured findings	The shape above	AI05 L4	Synthesis keeps each source
Error propagation	Simulated timeout gives type, query, partial results	AI06 L5	Report continues, and shows a coverage gap
Conflicts	Two credible sources disagree	AI05 L4	Both values kept and attributed

Failure table

What goes wrong in your exercise	What it teaches	TS
Hook passes when you expected a block	Matcher matches names, and the input has a different field	1.5
Rule never loads	Glob or scope is wrong	3.3
Skill floods the main session	No `context: fork`	3.2
Model returns invented values	Field was required	4.3
Retries don't converge	Data absent	4.4
Subagent output lacks facts	It never received them	1.3
Final report has no gap note	Empty results were treated as success	5.3

Java

The Java parts are in `code/ai08-java`: the agent loop, the MCP server, async batch handling and their tests. Only the model-calling paths are unrun. See AI08's references for what has been compiled and what's been run.

Trade-offs

Do Exercise 2 first if you only have time for one, because Domain 3 relies on small details, and candidates report it as weak even among daily users. That's my recommendation. Exercise 1 is next, since it covers Domain 1, the heaviest. **When not to build:** in the last two days, don't start any of these. Review your error log instead.

Common mistakes

Don't rule out an answer because of the scenario's label. The guide lists each scenario's primary domains, and that doesn't say the others can't appear.

> **Tip**
>
> Before each mock, reread that mock's four scenario tables. After it, mark which rows you got wrong. Those are your next study items.

Certification lens

> **Key idea**
>
> Supports **all 30 task statements** and all six scenarios. Scenario descriptions and exercises come from the guide, and the decision tables are this course's analysis of its objectives. The exercises weren't run end to end against a model.

These are original practice questions, not real or leaked.

Q Q1 · Exam style, scenario: Structured Data Extraction · Medium. Documents must complete within 30 hours, and batch processing can take up to 24 hours. What's the longest submission window that still guarantees the SLA?

A. 4 hours

B. 6 hours

C. 24 hours

D. 30 hours

A **Answer: B.** Window + 24 ≤ 30. The guide uses 4 hours, which leaves a margin, but the question asks for the longest.

Q Q2 · Exam style, scenario: Code Generation with Claude Code · Medium. Testing conventions should apply to all test files, which sit throughout the repository. Which is best?

A. A `CLAUDE.md` in each test directory

B. A `.claude/rules/` file with a `paths` glob

C. A section in `~/.claude/CLAUDE.md`

D. A slash command developers run first

A **Answer: B.** Glob-scoped rules apply by file type, regardless of directory.

Q Q3 · Exam style, scenario: Claude Code for Continuous Integration · Medium. Review comments repeat on every new commit. What's the best change?

A. Include prior findings, and ask for only new or unaddressed issues

B. Delete old comments before each run

C. Run reviews weekly through the batch API

D. Use a higher effort setting

A **Answer: A.** The model needs to know what's already been reported.

Q Q4 · Technique · Hard. If you could build only one guide exercise before the exam, and your weak area is Claude Code configuration, which one, and how would you prove each result?

A **Answer:** Exercise 2, confirming memory with `/memory`, rules by editing a matching and a non-matching file, forked skills by what enters the main context, and MCP scopes by listing both servers.

Revision sheet

→ S1: gates and hooks, escalation triggers, descriptions, case facts.

→ S2: scope, rules, skills, plan or direct, examples and interview.

→ S3: coordinator routes, explicit context, parallel calls, structured errors, provenance.

→ S4: Grep and Glob, Edit fallback, MCP scopes, resources, degradation, sessions.

→ S5: `-p` and JSON schema, criteria, independent review, multi-pass, prior findings, tempo.

→ S6: tool schema, nullable, validators and retry limits, batches by `custom_id`, segments.

→ SLA: window + batch time ≤ SLA.

→ Build Exercise 2 first, then Exercise 1, if time is short.

Your notes